PROJECT 1

Project1: Ubuntu – Developer -- Github – Jenkins -- AWS EC2

Contents

Cost-Controls.md

1. 1 ec2 t3micro → Used for demonstration → switched off when not using
2. 1 elastic IP currently stands as 13.41.43.163 accessible via http://
    - Guesstimate including t3micro cost is about £0.90 per 24 hours
3. Electricity at home for always on laptop/homelab
4.
5.

Lessons learnt

Having implemented a cicd pipeline, I learnt more details of what is involved in the setup:

1. I learnt more details about configuring Jenkins, and I am now practically more familiar with the UI
2. how to build a Jenkinsfile, which consists of stage and linux/bash commands
3. How to read the result of running the Jenkinsfile. SUCCESS at the bottom and the output build tet and package stage must match the input else pipeline would have read FAILED
4. How to build the pipeline in pieces and not be thinking of the cicd as an overwhelming whole one process that can be achieved in one stage/one sitting
5. The deployment process sits on one side of Jenkins, conceptually and github sits on the other side as the second stage in the process.
6. I automated the pull by Jenkins into github with SSH keys securing the 5 minute polling interval
7. AWS connection made from my home lab pc via Jenkins on home Ubuntu machine, with costs taken into consideration
8. Git and Linux are familiar and familiar from the command line. However, setting up git from Linux has been achieved for the first time, as opposed to using git UI in Windows/pycharm.
9. It is much easier to make necessary software changes from Linux, rather than the windows platform, eg, chmod 600 filexyz return. Job done
10. Requirements.txt
11. Practical experience of a test file and making change to it – changes must be exact -- Test_app.py
12. Build stage - To memory this was the package stage, compile all and tar -czf
13. Pull down clean up operation of Project1. Needed to be planned in advance. Lessons learnt for Project2.
14. Ed
15. Ed
16. Ed

PROGRESSION PLANS

Three projects will be completed in total to showcase my devops skills to employers with documentation, architecture diagram, costs documented, screenshots, troubleshooting, clean up / takedown of resources used. The technical details are documented where it is safe to write down, tests passed and failure scenarios tested. As I am doing a devops bootcamp, my plan is to implement projects as I learn a subject and not be linear with all theory done before putting it into practice. The final portfolio

By the end, I want an employer to be able to click my GitHub profile and see:

```
Cornel's DevOps Portfolio
│
├── Project 1
│   └── CI/CD → AWS EC2 + Devops learning Journal
│
├── Project 2
│   └── Secure CI/CD → Nexus → AWS  + Devops learning Journal
│
├── Project 3
│   └── Kubernetes DevOps Platform -  + Devops learning Journal
│
└── DevOps Learning Journal
```

And the progression is immediately obvious:

```
PROJECT 1
"I can build CI/CD."

       ↓

PROJECT 2
"I can build secure CI/CD."

       ↓

PROJECT 3
"I can build and operate a containerized
Kubernetes deployment platform."
```

That's the story I'd want my CV and GitHub profile to tell.

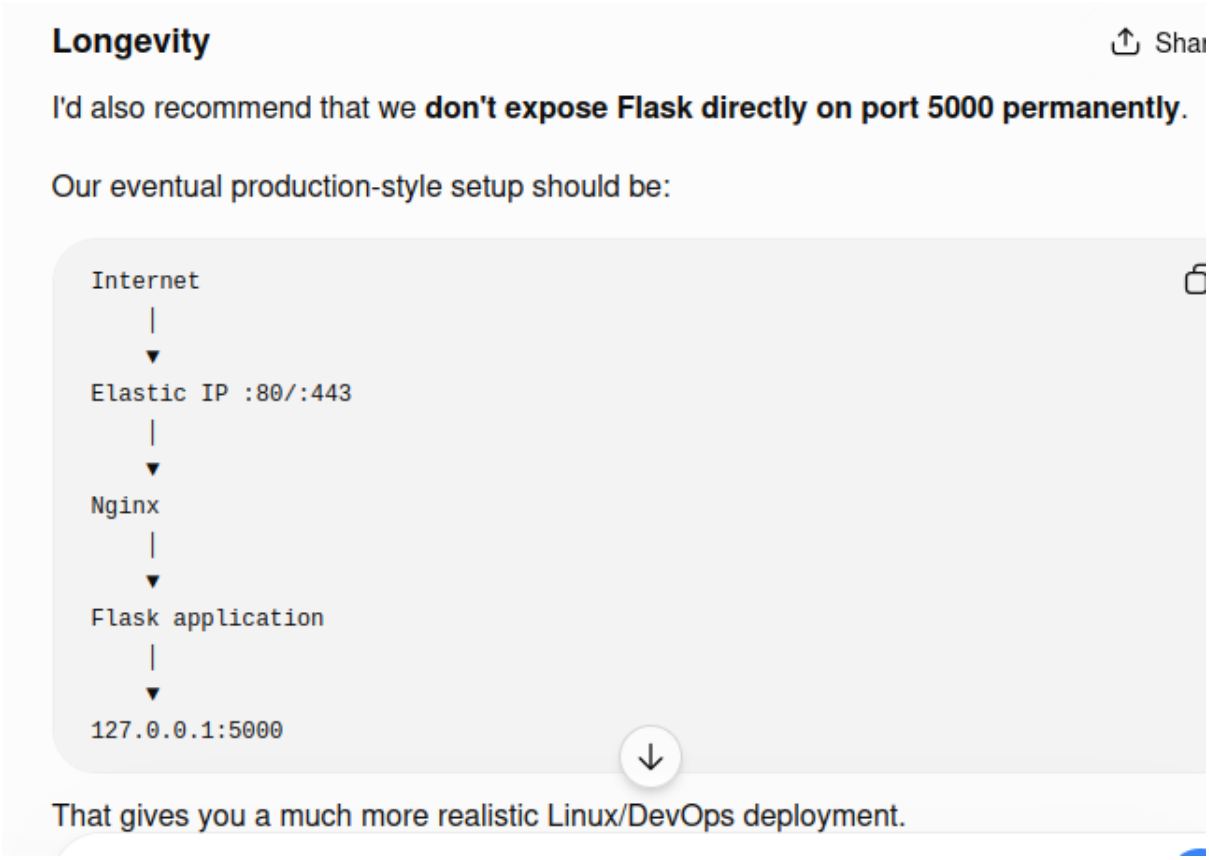
And importantly, **don't wait until all 200 hours are finished to start applying for jobs**. Once Project 1 is polished, you can already use it as evidence in applications. Project 2 then strengthens your profile, and Project 3 becomes the flagship piece.

If you're doing roughly **10–15 hours a week**, I'd regard **3–4 months** as a very sensible target for the complete three-project portfolio, while continuing the bootcamp alongside it.

Troubleshooting

1. 3 to 4 terminals open, I would accidentally use the wrong one
2. small window space to work in so switching, chopping and changing constantly between terminals, Jenkins, chatgpt and Github
3. Dubious mouse
4. Whats my IP /32 changed as ssh was no longer working
5. Brower firefox in Ubuntu, bidirection cut, copy and paste, not working, bidirection copy to host windows not operational, so I have to use snip tool instead to record and try to produce reproducible output and screen shots
6. Occasionally, at times, chatgpt was not as precise with instructions. Maybe for example due to a newer version of Jenkins, I was left to figure out stuff for myself while filling in form details and other critical variables, such as setting up Jenkins connection to Github / AWS. The order of instructions were not ideal and far from seamless. Small endless frustrations.
7. My keyboard on laptop – wacky, o, s, r, t, c only works sometimes
8. No reproducible commands structure was preserved, as chatgpt was asking for a pasting of output of commands while I was going along. In project2, I will try to reproduce the output

Screenshots



That gives you a much more realistic Linux/DevOps deployment.

And eventually:

```
GitHub
   ↓
Jenkins
   ↓
Build
   ↓
Test
   ↓
Package
   ↓
Deploy
   ↓
EC2
   ↓
Nginx
   ↓
Flask
```

That's a **portfolio-quality CI/CD exercise**, ↓ er than just a Jenkins tutorial.

What Build #13 proved

Your pipeline successfully completed:

1. **GitHub checkout** — Jenkins retrieved commit `02e18ca`.
2. **Build** — Python compiled successfully.
3. **Test** — `pytest` passed: **1 passed**.
4. **System Information** — Jenkins confirmed it is running as `jenkins`.
5. **Package** — `jenkins-demo.tar.gz` was created.
6. **EC2 SSH** — Jenkins connected to your EC2 instance:

```
ubuntu
```

7. **Artifact archive** — Jenkins archived the package.
8. Final result:

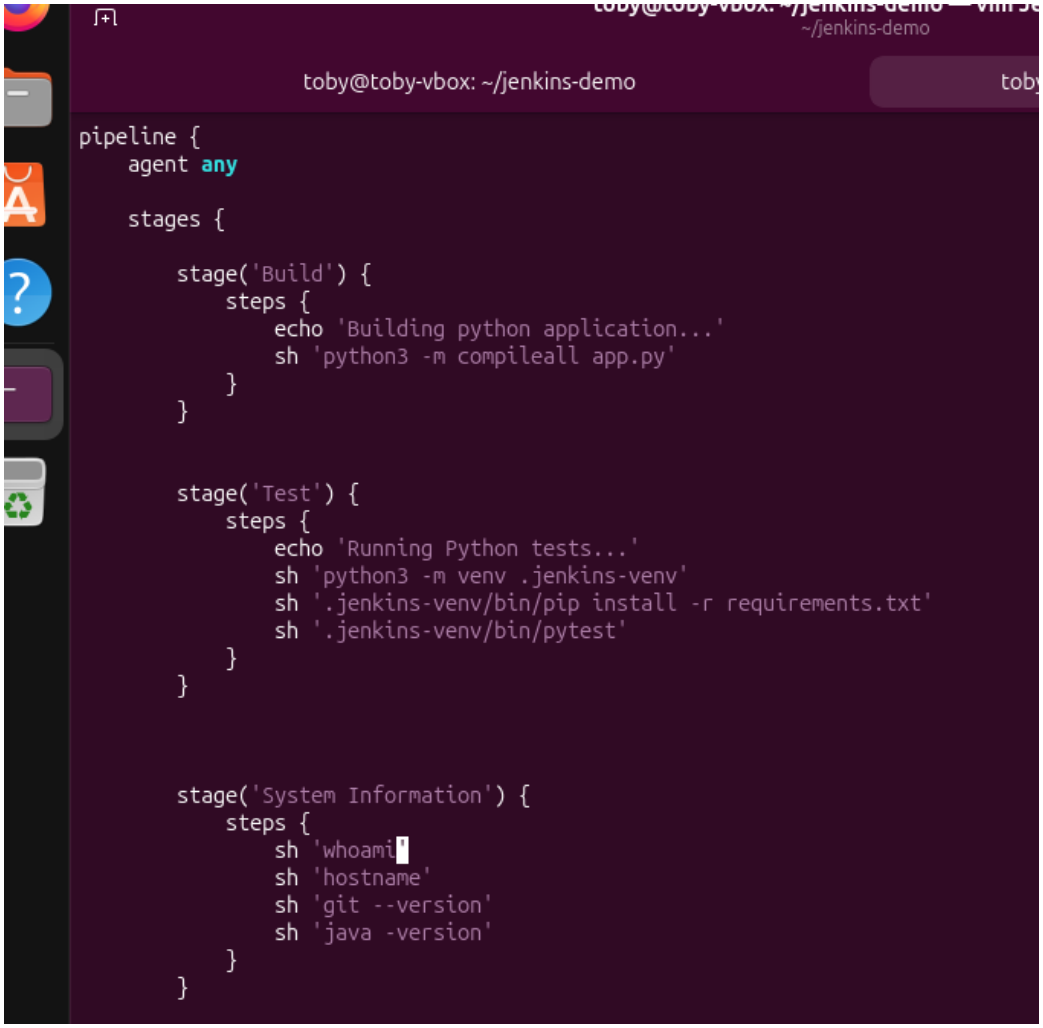
```
Finished: SUCCESS
```

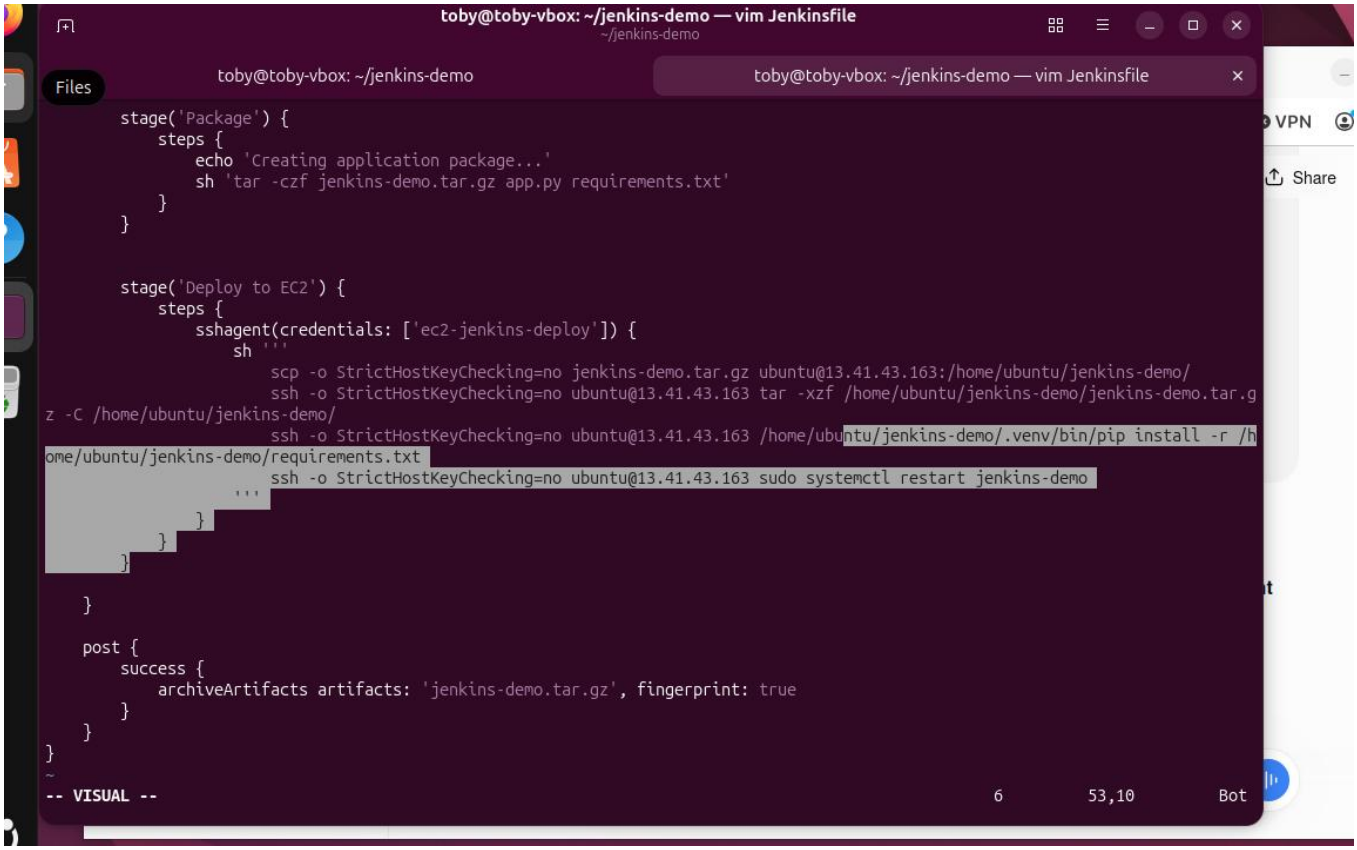
Your complete pipeline

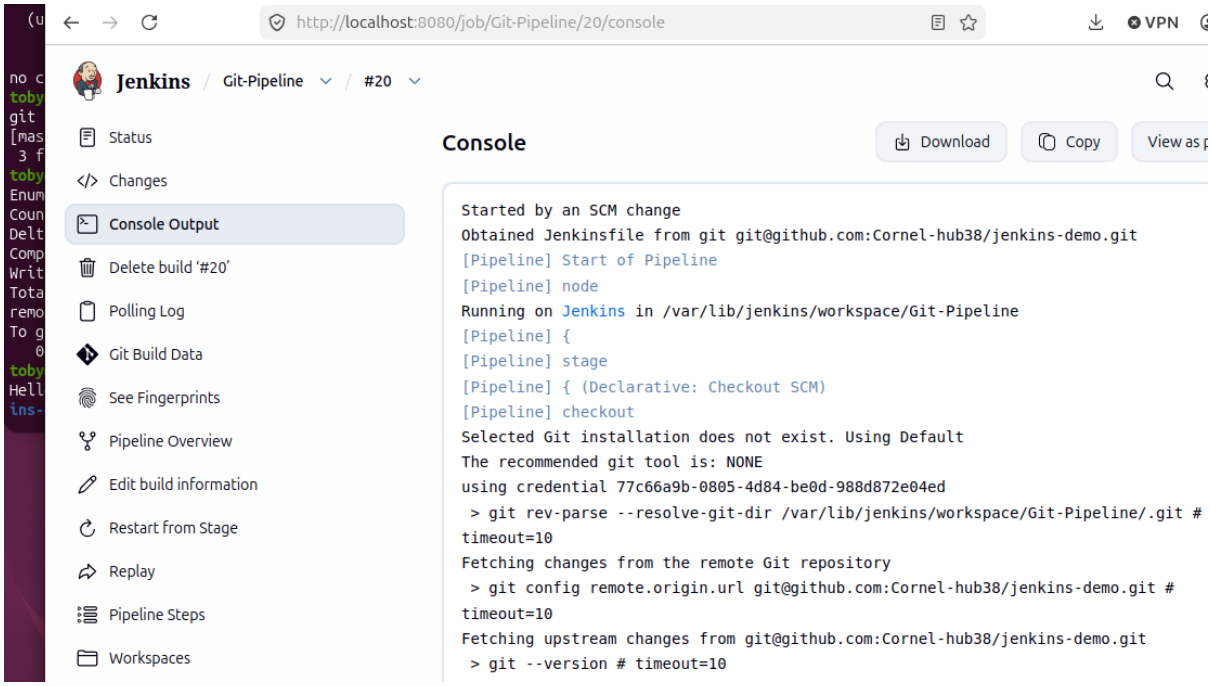
```
Developer
    |
    | git push
    ▼
GitHub
    |
    | SCM polling
    ▼
Jenkins
    |
    ├── Build ✅
    ├── Test ✅ 1 passed
    ├── Package ✅
    |
    ▼
EC2 Deployment
    |
    ├── SCP package ✅
    ├── Extract ✅
    ├── Install dependencies ✅
    ├── Restart Gunicorn ✅
    |
    ▼
Nginx :80
    |
    ▼
Flask application
    |
    ▼
test2 auto ✅
```

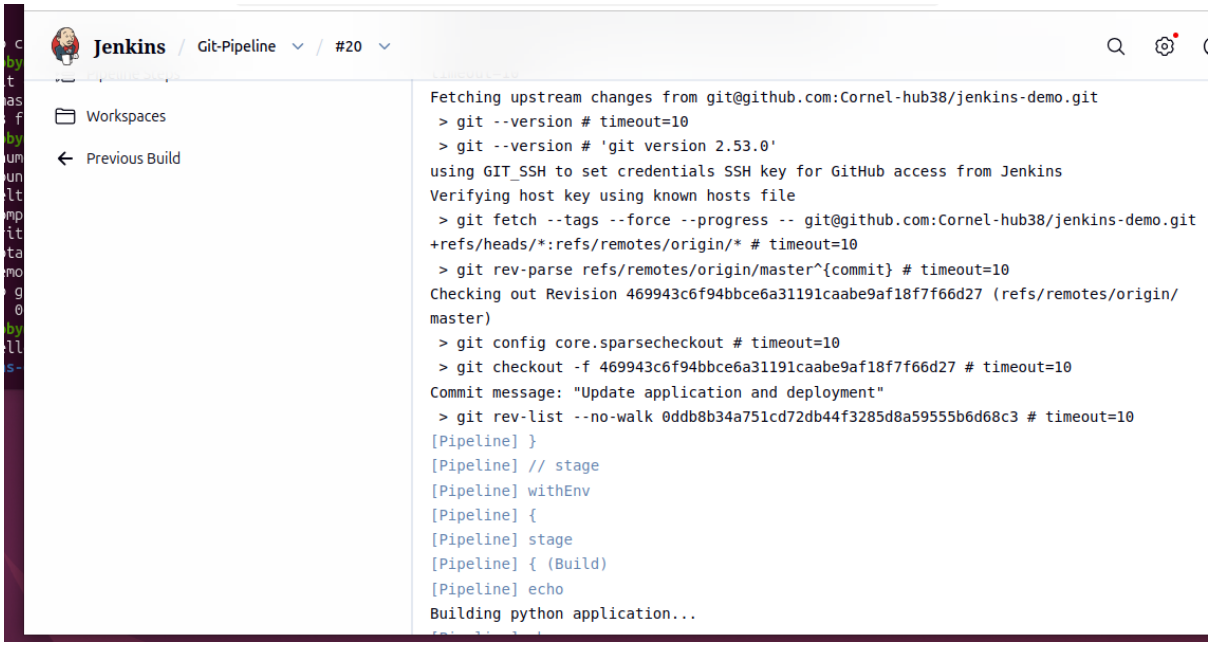
This is now a **genuine working CI/CD demonstration**, not just a Jenkins tutorial exercise.

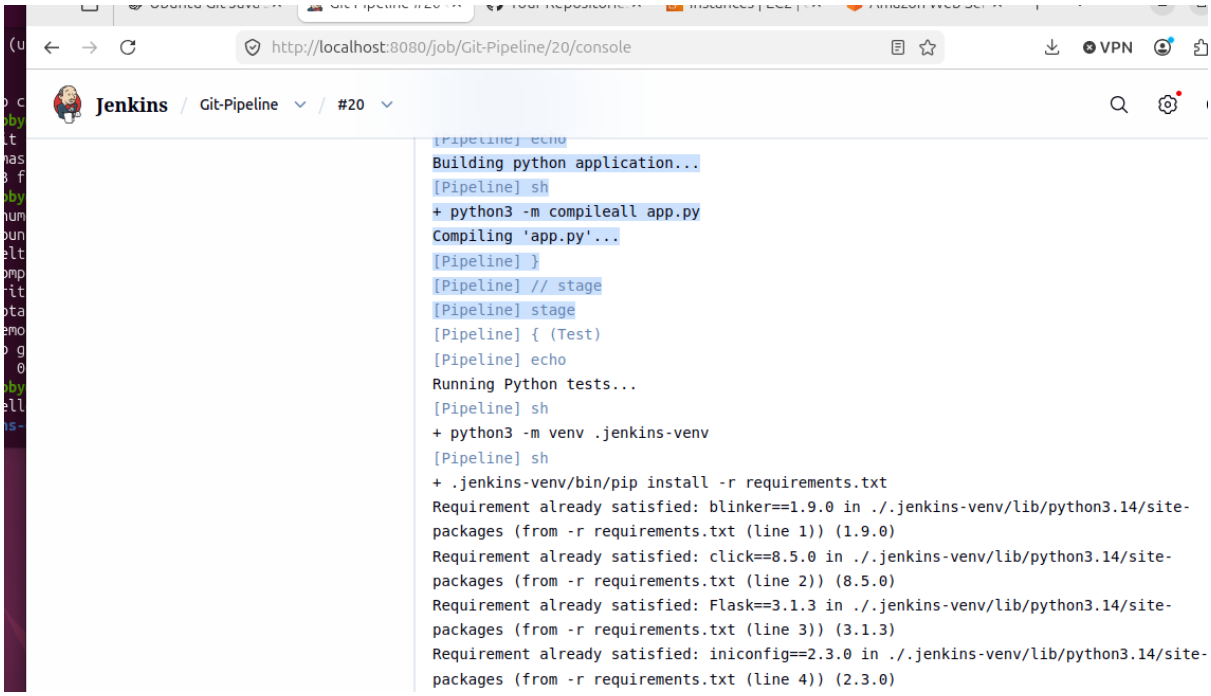
One small improvement I'd recommend next is addi automated **post-deployment health check** to Jenkins. That way Jenkins won't merely restart the application—it will actually test the live endpoint and

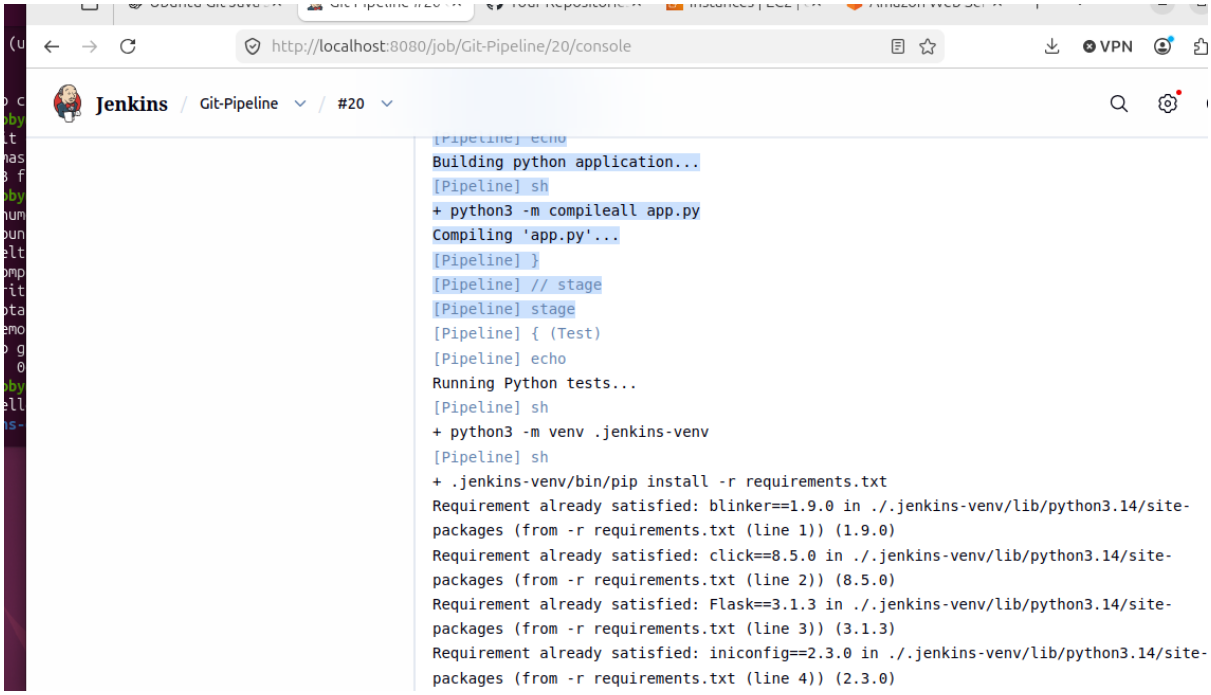

~/jenkins-demo
toby@toby-vbox: ~/jenkins-demo
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building python application...'
sh 'python3 -m compileall app.py'
}
}
stage('Test') {
steps {
echo 'Running Python tests...'
sh 'python3 -m venv .jenkins-venv'
sh '.jenkins-venv/bin/pip install -r requirements.txt'
sh '.jenkins-venv/bin/pytest'
}
}
stage('System Information') {
steps {
sh 'whoami'
sh 'hostname'
sh 'git --version'
sh 'java -version'
}
}

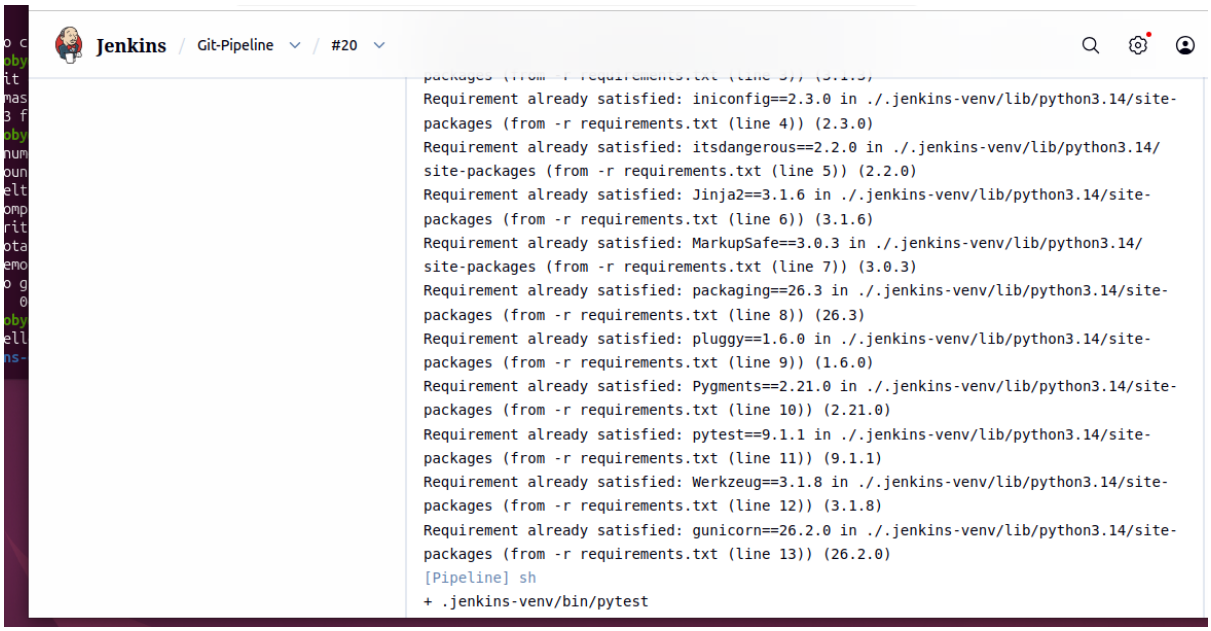

toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
~/jenkins-demo
Files
toby@toby-vbox: ~/jenkins-demo
toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
stage('Package') {
steps {
echo 'Creating application package...'
sh 'tar -czf jenkins-demo.tar.gz app.py requirements.txt'
}
}
stage('Deploy to EC2') {
steps {
sshagent(credentials: ['ec2-jenkins-deploy']) {
sh '''
scp -o StrictHostKeyChecking=no jenkins-demo.tar.gz ubuntu@13.41.43.163:/home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 tar -xzf /home/ubuntu/jenkins-demo/jenkins-demo.tar.gz -C /home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 /home/ubuntu/jenkins-demo/.venv/bin/pip install -r /home/ubuntu/jenkins-demo/requirements.txt
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 sudo systemctl restart jenkins-demo
'''
}
}
}
}
post {
success {
archiveArtifacts artifacts: 'jenkins-demo.tar.gz', fingerprint: true
}
}
}
-- VISUAL --
6
53,10
Bot
VPN
Share

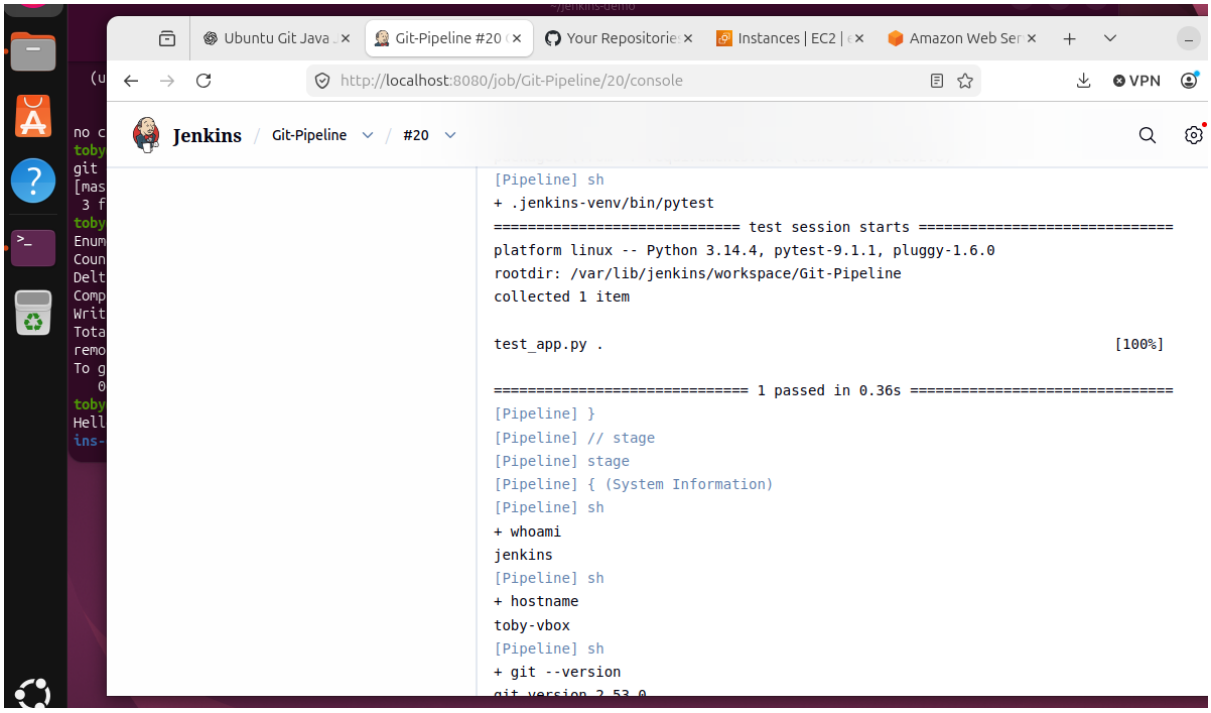

http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins
Git-Pipeline
#20
Status
Changes
Console Output
Delete build '#20'
Polling Log
Git Build Data
See Fingerprints
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Console
Download
Copy
View as p
Started by an SCM change
Obtained Jenkinsfile from git git@github.com:Cornel-hub38/jenkins-demo.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Git-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential 77c66a9b-0805-4d84-be0d-988d872e04ed
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Git-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url git@github.com:Cornel-hub38/jenkins-demo.git # timeout=10
Fetching upstream changes from git@github.com:Cornel-hub38/jenkins-demo.git
> git --version # timeout=10

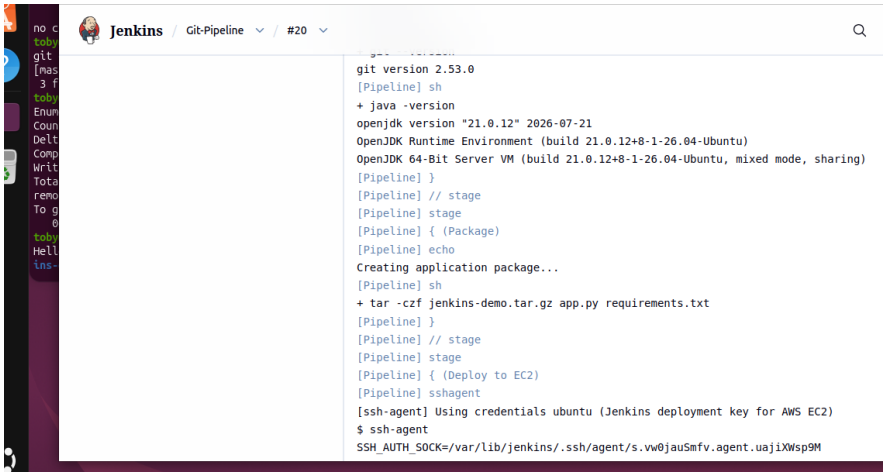

Jenkins / Git-Pipeline / #20
Workspaces
Previous Build
Fetching upstream changes from git@github.com:Cornel-hub38/jenkins-demo.git
> git --version # timeout=10
> git --version # 'git version 2.53.0'
using GIT_SSH to set credentials SSH key for GitHub access from Jenkins
Verifying host key using known hosts file
> git fetch --tags --force --progress -- git@github.com:Cornel-hub38/jenkins-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision 469943c6f94bbce6a31191caabe9af18f7f66d27 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 469943c6f94bbce6a31191caabe9af18f7f66d27 # timeout=10
Commit message: "Update application and deployment"
> git rev-list --no-walk 0ddb8b34a751cd72db44f3285d8a59555b6d68c3 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building python application...

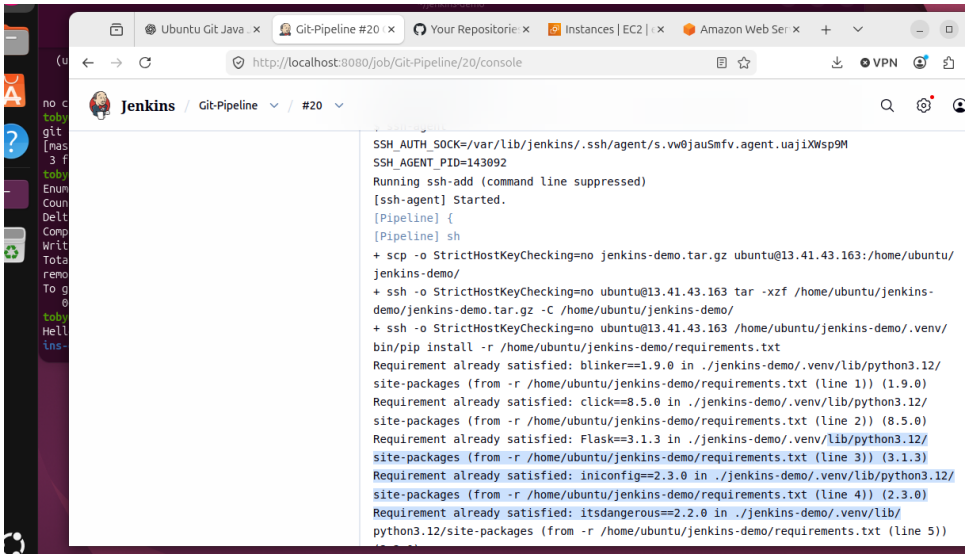

http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins / Git-Pipeline / #20
Building python application...
[Pipeline] sh
+ python3 -m compileall app.py
Compiling 'app.py'...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running Python tests...
[Pipeline] sh
+ python3 -m venv .jenkins-venv
[Pipeline] sh
+ .jenkins-venv/bin/pip install -r requirements.txt
Requirement already satisfied: blinker==1.9.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: click==8.5.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 2)) (8.5.0)
Requirement already satisfied: Flask==3.1.3 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 3)) (3.1.3)
Requirement already satisfied: iniconfig==2.3.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 4)) (2.3.0)

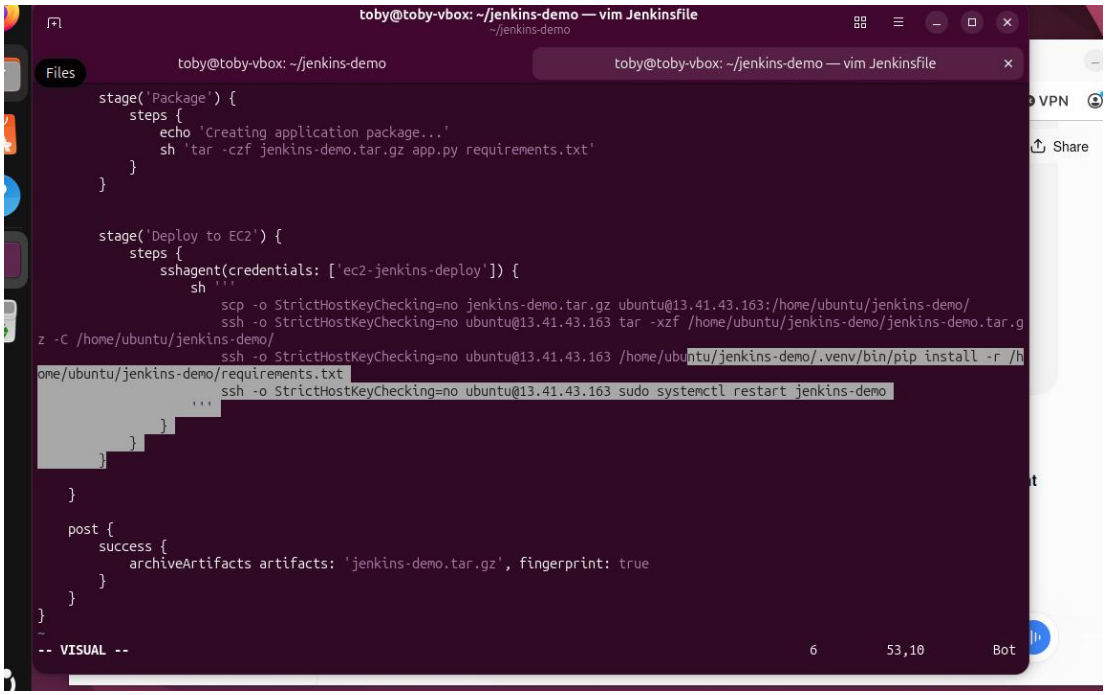

http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins
Git-Pipeline
#20
Building python application...
[Pipeline] sh
+ python3 -m compileall app.py
Compiling 'app.py'...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running Python tests...
[Pipeline] sh
+ python3 -m venv .jenkins-venv
[Pipeline] sh
+ .jenkins-venv/bin/pip install -r requirements.txt
Requirement already satisfied: blinker==1.9.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: click==8.5.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 2)) (8.5.0)
Requirement already satisfied: Flask==3.1.3 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 3)) (3.1.3)
Requirement already satisfied: iniconfig==2.3.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 4)) (2.3.0)

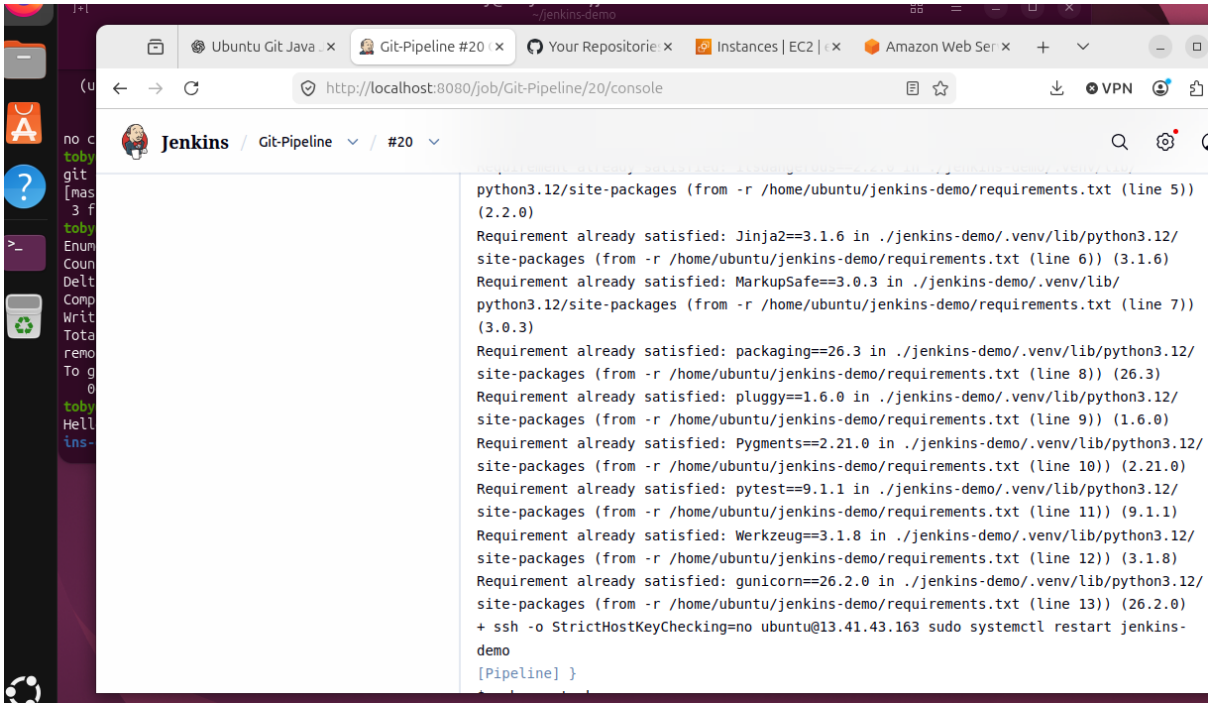

Jenkins
Git-Pipeline
#20
Requirement already satisfied: iniconfig==2.3.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 4)) (2.3.0)
Requirement already satisfied: itsdangerous==2.2.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 5)) (2.2.0)
Requirement already satisfied: Jinja2==3.1.6 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 6)) (3.1.6)
Requirement already satisfied: MarkupSafe==3.0.3 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 7)) (3.0.3)
Requirement already satisfied: packaging==26.3 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 8)) (26.3)
Requirement already satisfied: pluggy==1.6.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 9)) (1.6.0)
Requirement already satisfied: Pygments==2.21.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 10)) (2.21.0)
Requirement already satisfied: pytest==9.1.1 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 11)) (9.1.1)
Requirement already satisfied: Werkzeug==3.1.8 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 12)) (3.1.8)
Requirement already satisfied: gunicorn==26.2.0 in ./.jenkins-venv/lib/python3.14/site-packages (from -r requirements.txt (line 13)) (26.2.0)
[Pipeline] sh
+ .jenkins-venv/bin/pytest

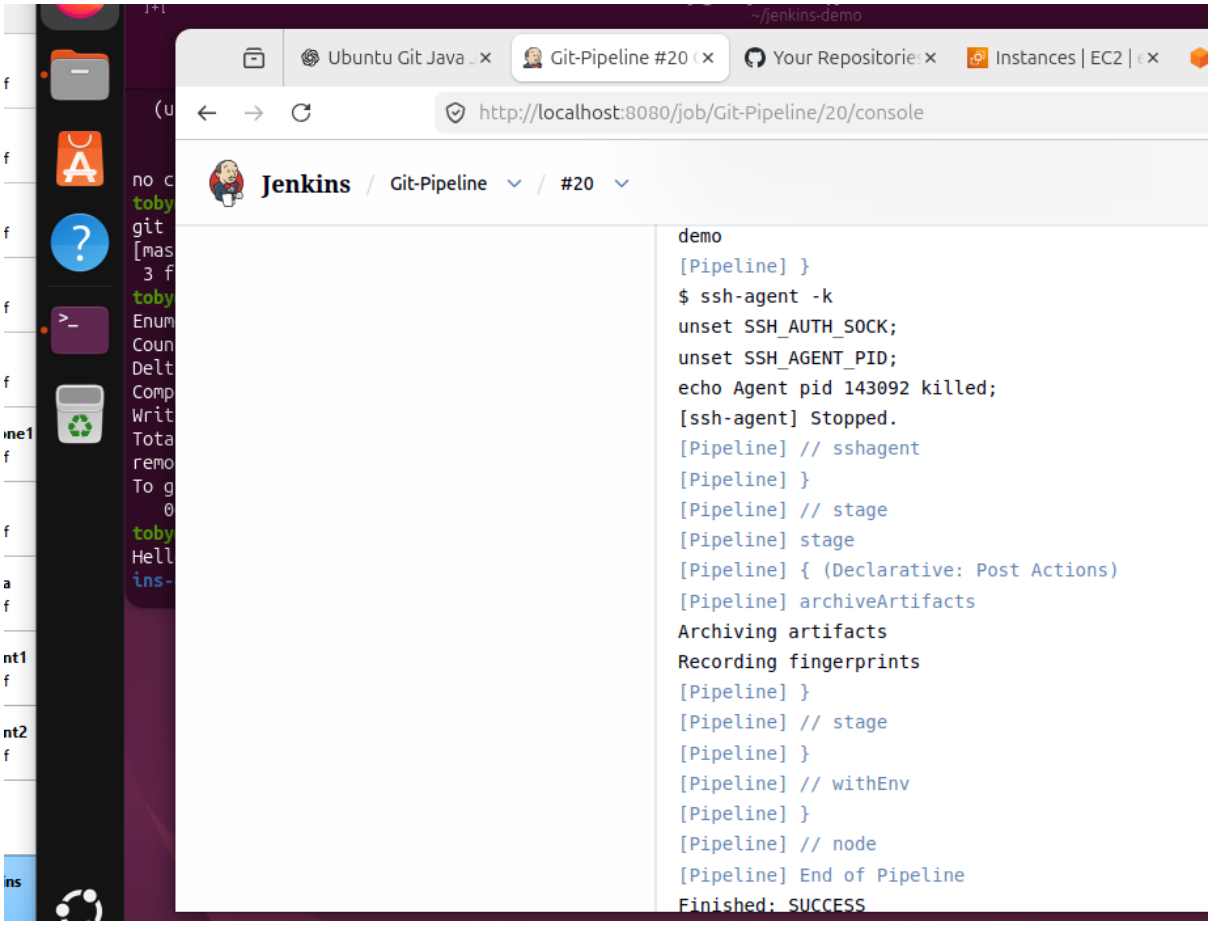

Ubuntu Git Java
Git-Pipeline #20
Your Repositorie
Instances | EC2 |
Amazon Web Ser
http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins / Git-Pipeline / #20
[Pipeline] sh
+ .jenkins-venv/bin/pytest
============================= test session starts ==============================
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: /var/lib/jenkins/workspace/Git-Pipeline
collected 1 item
test_app.py . [100%]
============================== 1 passed in 0.36s ===============================
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (System Information)
[Pipeline] sh
+ whoami
jenkins
[Pipeline] sh
+ hostname
toby-vbox
[Pipeline] sh
+ git --version
git version 2.53.0

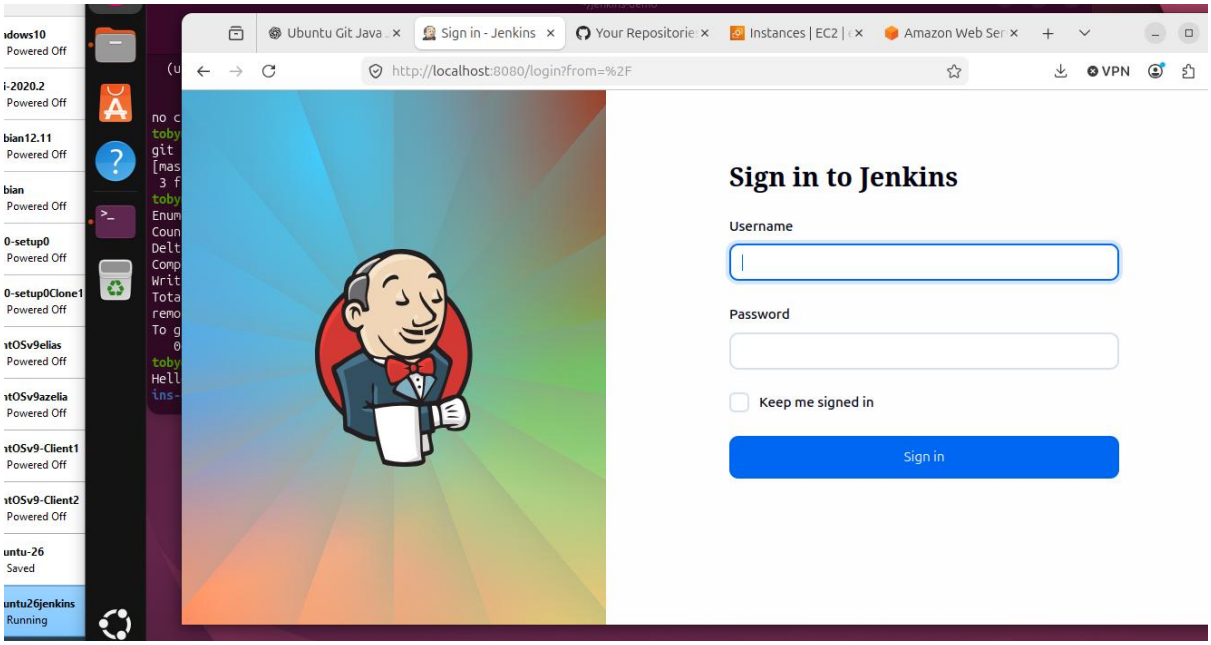

Jenkins / Git-Pipeline / #20
git version 2.53.0
[Pipeline] sh
+ java -version
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-26.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-26.04-Ubuntu, mixed mode, sharing)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Package)
[Pipeline] echo
Creating application package...
[Pipeline] sh
+ tar -czf jenkins-demo.tar.gz app.py requirements.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to EC2)
[Pipeline] sshagent
[ssh-agent] Using credentials ubuntu (Jenkins deployment key for AWS EC2)
$ ssh-agent
SSH_AUTH_SOCK=/var/lib/jenkins/.ssh/agent/s.vw0jauSmfv.agent.uajiXWsp9M

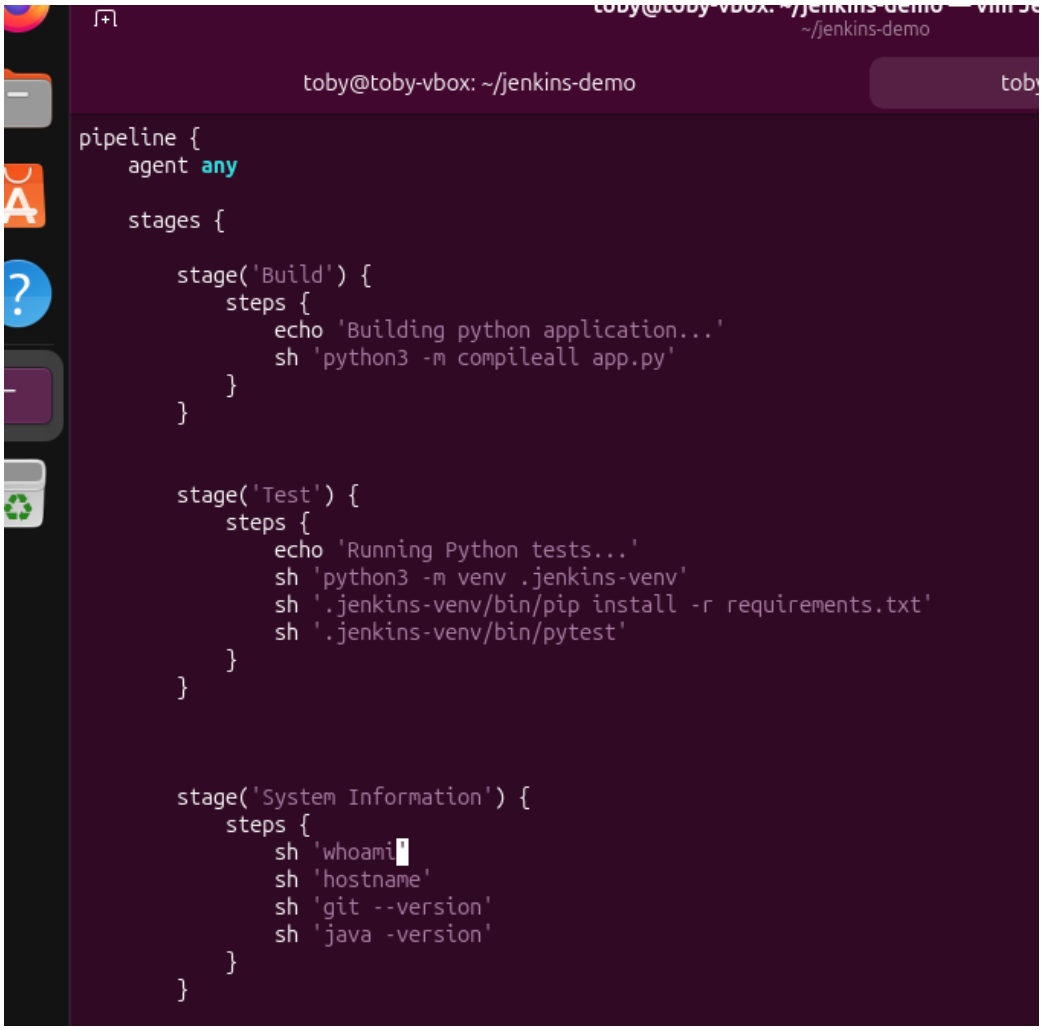

Ubuntu Git Java
Git-Pipeline #20
Your Repositorie
Instances | EC2 |
Amazon Web Ser
http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins
Git-Pipeline
#20
SSH_AUTH_SOCK=/var/lib/jenkins/.ssh/agent/s.vw0jauSmfv.agent.uajiXWsp9M
SSH_AGENT_PID=143092
Running ssh-add (command line suppressed)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ scp -o StrictHostKeyChecking=no jenkins-demo.tar.gz ubuntu@13.41.43.163:/home/ubuntu/
jenkins-demo/
+ ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 tar -xzf /home/ubuntu/jenkins-
demo/jenkins-demo.tar.gz -C /home/ubuntu/jenkins-demo/
+ ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 /home/ubuntu/jenkins-demo/.venv/
bin/pip install -r /home/ubuntu/jenkins-demo/requirements.txt
Requirement already satisfied: blinker==1.9.0 in ./jenkins-demo/.venv/lib/python3.12/
site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: click==8.5.0 in ./jenkins-demo/.venv/lib/python3.12/
site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 2)) (8.5.0)
Requirement already satisfied: Flask==3.1.3 in ./jenkins-demo/.venv/lib/python3.12/
site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 3)) (3.1.3)
Requirement already satisfied: iniconfig==2.3.0 in ./jenkins-demo/.venv/lib/python3.12/
site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 4)) (2.3.0)
Requirement already satisfied: itsdangerous==2.2.0 in ./jenkins-demo/.venv/lib/
python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 5))

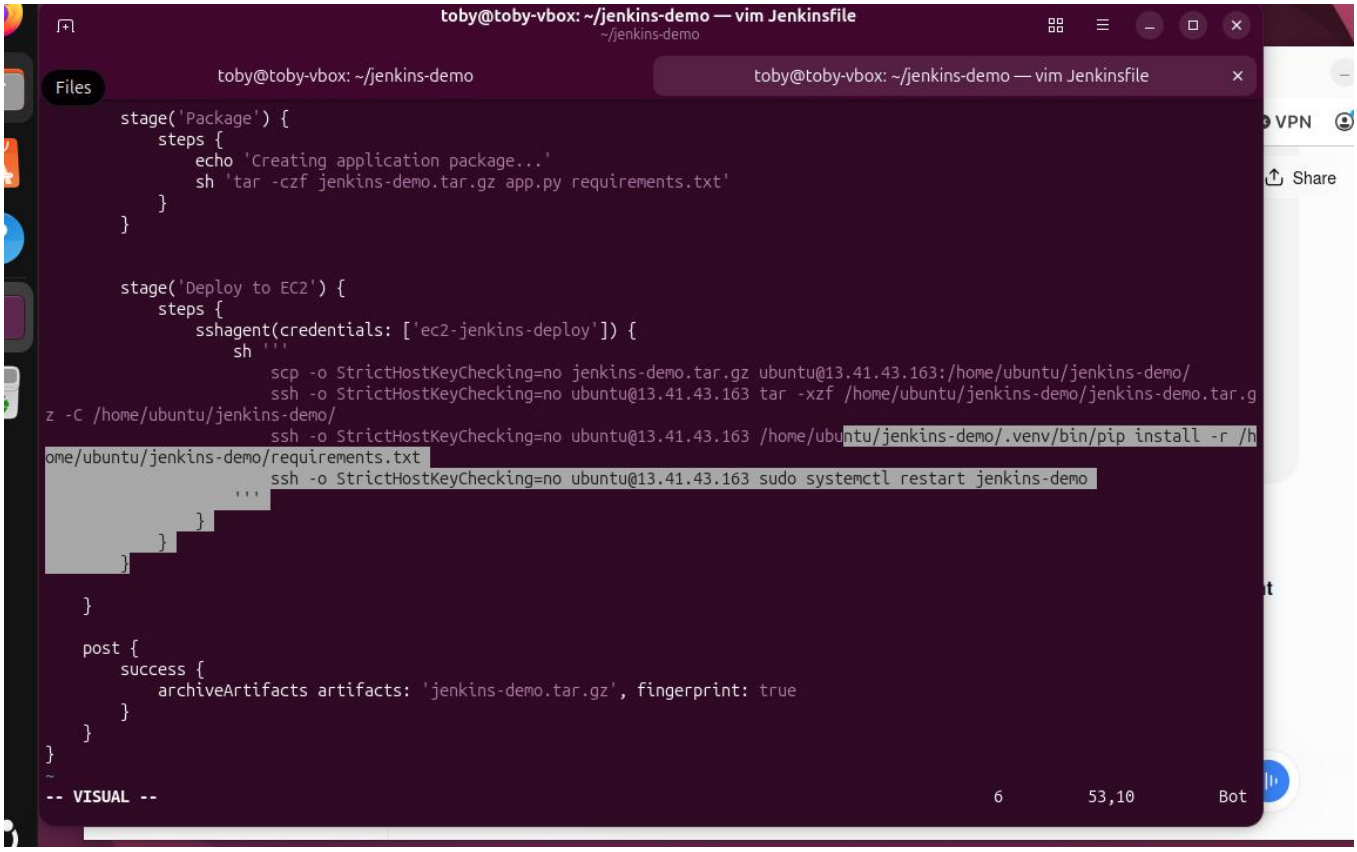

toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
~/jenkins-demo
Files
toby@toby-vbox: ~/jenkins-demo
toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
VPN
Share
stage('Package') {
steps {
echo 'Creating application package...'
sh 'tar -czf jenkins-demo.tar.gz app.py requirements.txt'
}
}
stage('Deploy to EC2') {
steps {
sshagent(credentials: ['ec2-jenkins-deploy']) {
sh '''
scp -o StrictHostKeyChecking=no jenkins-demo.tar.gz ubuntu@13.41.43.163:/home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 tar -xzf /home/ubuntu/jenkins-demo/jenkins-demo.tar.g
z -C /home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 /home/ubuntu/jenkins-demo/.venv/bin/pip install -r /h
ome/ubuntu/jenkins-demo/requirements.txt
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 sudo systemctl restart jenkins-demo
'''
}
}
}
}
post {
success {
archiveArtifacts artifacts: 'jenkins-demo.tar.gz', fingerprint: true
}
}
}
-- VISUAL --
6
53,10
Bot


Ubuntu Git Java
Git-Pipeline #20
Your Repositories
Instances | EC2 |
Amazon Web Ser
http://localhost:8080/job/Git-Pipeline/20/console
VPN
Jenkins
Git-Pipeline
#20
python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 5)) (2.2.0)
Requirement already satisfied: Jinja2==3.1.6 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 6)) (3.1.6)
Requirement already satisfied: MarkupSafe==3.0.3 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 7)) (3.0.3)
Requirement already satisfied: packaging==26.3 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 8)) (26.3)
Requirement already satisfied: pluggy==1.6.0 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 9)) (1.6.0)
Requirement already satisfied: Pygments==2.21.0 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 10)) (2.21.0)
Requirement already satisfied: pytest==9.1.1 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 11)) (9.1.1)
Requirement already satisfied: Werkzeug==3.1.8 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 12)) (3.1.8)
Requirement already satisfied: gunicorn==26.2.0 in ./jenkins-demo/.venv/lib/python3.12/site-packages (from -r /home/ubuntu/jenkins-demo/requirements.txt (line 13)) (26.2.0)
+ ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 sudo systemctl restart jenkins-demo
[Pipeline] }


Ubuntu Git Java
Git-Pipeline #20
Your Repositories
Instances | EC2 |
http://localhost:8080/job/Git-Pipeline/20/console
Jenkins
Git-Pipeline
#20
demo
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 143092 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
Archiving artifacts
Recording fingerprints
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Ubuntu Git Java
Sign in - Jenkins
Your Repositorie
Instances | EC2 |
Amazon Web Ser
http://localhost:8080/login?from=%2F
VPN
Sign in to Jenkins
Username
Password
Keep me signed in
Sign in


toby@toby-vbox: ~/jenkins-demo
~/jenkins-demo
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building python application...'
sh 'python3 -m compileall app.py'
}
}
stage('Test') {
steps {
echo 'Running Python tests...'
sh 'python3 -m venv .jenkins-venv'
sh '.jenkins-venv/bin/pip install -r requirements.txt'
sh '.jenkins-venv/bin/pytest'
}
}
stage('System Information') {
steps {
sh 'whoami'
sh 'hostname'
sh 'git --version'
sh 'java -version'
}
}


toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
~/jenkins-demo
Files
toby@toby-vbox: ~/jenkins-demo
toby@toby-vbox: ~/jenkins-demo — vim Jenkinsfile
stage('Package') {
steps {
echo 'Creating application package...'
sh 'tar -czf jenkins-demo.tar.gz app.py requirements.txt'
}
}
stage('Deploy to EC2') {
steps {
sshagent(credentials: ['ec2-jenkins-deploy']) {
sh '''
scp -o StrictHostKeyChecking=no jenkins-demo.tar.gz ubuntu@13.41.43.163:/home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 tar -xzf /home/ubuntu/jenkins-demo/jenkins-demo.tar.gz -C /home/ubuntu/jenkins-demo/
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 /home/ubuntu/jenkins-demo/.venv/bin/pip install -r /home/ubuntu/jenkins-demo/requirements.txt
ssh -o StrictHostKeyChecking=no ubuntu@13.41.43.163 sudo systemctl restart jenkins-demo
'''
}
}
}
}
post {
success {
archiveArtifacts artifacts: 'jenkins-demo.tar.gz', fingerprint: true
}
}
}
-- VISUAL --
6
53,10
Bot
VPN
Share

Architecture diagram

4. Project 1 — Basic CI/CD

I'd now formally define your first project as:

Project 1 — Automated CI/CD Deployment to AWS

Technologies

```
Linux
Git
GitHub
Jenkins
Docker
AWS EC2
```

Architecture:

```
Developer
    |
    ▼
 GitHub
    |
    | webhook
    ▼
 Jenkins
    |
    ├── Checkout
    ├── Build
    ├── Test
    └── Package
         |
         ▼
       Docker
         |
         ▼
       AWS EC2
         |
         ▼
     Application
```

High Level Design Breakdown – 60% Complete

1. Update Ubuntu
2. Install Git
3. Install Java
4. Install Jenkins
5. Start Jenkins
    a. Open Jenkins
    b. Get the Jenkins administrator password
    c. Create a simple Jenkins pipeline
    d. Run the pipeline
    e. Put the pipeline into Git
    f. Create a project directory
    g. Create the Jenkinsfile
    h. Check the file
    i.
6. Initialize Git
    a. Add and comit the Jenkinsfile
    b. Make your first commit
    c. Verify the commit
7. Connect Jenkins to the Git repository
    a. Create a new Jenkins Pipeline
    b. Configure the pipeline
    c. Make a Jenkins-accessible Git Repository
    d. Configure Jenkins t use Git
    e. Run it – the pipeline
    f. Build now
    g. Console output
    h. Diagnose Build 1
    i. Allow th local Git checkout
    j. Reload Systemd
    k. Add the Jenkins Git setting via nano/vim
    l. Verify the setting
    m. Add the JVM option correctly
    n. Use Jenkins system property directly
    o.
8. Put the repository on Github
    a. Create the Github repository
    b. Connect your local repository to GitHub
    c. Push your existing commit
    d. Create an SSH key
        i. Copy your public key
        ii. Change the Git remote from HTTPS to SSH
    e. Push the Jenkinsfile
9. Configure Jenkins to access GitHub
    a. Generate a Jenkinss SSH Key

b. Display Jenkins public key
c. Add it to GitHub
d. Test GitHub SSH access as Jenkins
e. Add the Jenkins SSH key as a Jenkins credential
    i. Open Jenkins → http://localhost:8080
    ii. Manage Jenkins credentials
    iii. Stores scoped to Jenkins
    iv. Jenkins
    v.

f. Create the credential
    i. Kind
    ii. Scope
    iii. username
g. configure Git-Pipeline
    i. definition
    ii. scm
    iii. repository URL
    iv. credentials
    v. branch specifier
    vi. Script path
h. Build the pipeline with Build now

10. Add a real application/build stage
    a. Add build and tet stages
    b. Modify the Jenkinsfile locally
    c. Check the file
    d. Git commit
        i. commit
    e. Push to GitHub
    f. Run the updated Jenkins pipeline
    g. Get the FAILURE
    h. Correct the Jenkins job
    i. Run Build 3
        i. Console output
11. Preserve the package as a Jenkins artifact
    a. Edit the Jenkinsfile
    b. Check it
    c. Commit the change
    d. Push to GitHub
    e. Run Build in Jenkins tab
    f.
12. GitHub – Jenkins automatic triggering
    a. Choice = a webhook + tunnel or polling every 5 minutes
    b. Enable Jenkins Polling
        i. Open the Jenkins Job configuration
        ii. Build Triggers
        iii. POLL SCM
        iv. A schedule text box appears
    c. Save
    d. Create a change
    e. Commit the change

f. Push the change to GitHub
g. Wait for Jenkins to detect the change
h. SCM Polling is working properly as evidenced by “Started by an SCM change”
i.

13. WHERE WOULD WE DEPLOY IT? EC2
    a. Python is the choosen language
    b. Install thee python venv package
    c. Create the virtual environment
    d. Activate the virtual environment
    e. Install Flask
    f. Create the application
    g. Start the flask application locally
    h. Leave the terminal running
    i. Test it locally
    j. Stop the flask development server
    k. Create the requirements.txt
    l. Check our project files
    m. Create .gitignore
    n. Check what git sees
    o. Add the 3 files to git
    p. Commit the application
    q. Push the application to gitHub
    r. Let Jenkins detect the new application
    s. Modify the Jenkins pipeline
        i. Create test_app.py
        ii.
    t. Install pytest
    u. Run the test locally
    v. Add pytest to requirements.txt
    w. Commit the test and update requirements
    x. Stage the two files
    y. Commit them
    z. Push the test files
    aa. Modify the Jenkinsfile
    bb. Commit the Jenkinsfile
    cc. Stage the Jenkins file
    dd. Commit it
    ee. Push the Jenkinsfile to GitHub
    ff. Let SCM polling trigger Jenkins
    gg. This is now a genuine CI pipeline
    hh. Make the build stage real
    ii. 12.10b
14. F
15. F
16. F
17. F
18. F

INSTALLATION Instructions

1. If you look under troubleshooting you will see that implementation was a cumbersome, time consuming exercise. Thus only the #history commands are left and would not be an accurate picture of a reproducible project for project 1. See High Level Design Breakdown above for part of reproducing Project1. This means that project2 will be similarly awkward to document the reproducible aspects of the project

Deployment instructions for presenting demo to employer

1. Pwd = /home/tobijones; cd Jenkins-demo; ls
2. On home lab Ubuntu, Make a small textual change to app.py
3. Reflect that exact change in test_app.py the pytest file
4. At the Ubuntu console, type Git status
5. Git add .
6. Git status
7. # git commit –m “message”
8. # git push origin master
9. That will automate the cicd process, starting from that push
10. # remember the 5 minute SCM polling of github servers for changes in code
11. Open Jenkins locally on home lab laptop at
12. Build History
13. Console output
14. Refresh webpage to update to changes in app.py file
15. Check AWS ec2 instance for reflected changes once refreshed SCM polling has taken place

Connecting Jenkins to github

Once Jenkins was installed tested and the Jenkinsfile built, it needed to be connected to git hub. Chatgpt gave me a choice of :

1. Build a tunnel between Jenkins and Github using SSH and webhooks
2. Poll github every 5, 10 minutes using an SSH connection using SCM plling.

I choose option 2 as it appeared the safest approach, and simplest to implement.

SSH key gen was used to generate the keys. And the built approach to the entire project was incremental. Cost, security were paramount factors.

Connecting Jenkins to AWS EC2

Automating pipeline as of git push command

See above, section called High Level Design Breakdown

Project1 completion checklist implementation

Each project of project1, project2, project3 should finish only when you've got:

Technical

9. [x] Application works
10. [x] Git repository works
11. [x] Jenkins pipeline works
12. [x] Automated deployment works
13. [ ] Infrastructure reproducible
14. [ ] Security controls implemented
15. [x] Tests pass
16. [ ] Failure scenarios tested
17. [x] Ec2 works

Documentation

18. [ ] README
19. [x] Architecture diagram
20. [x] Installation instructions
21. [x] Deployment instructions
22. [ ] Security documentation
23. [x] Cost documentation
24. [x] Troubleshooting
25. [x] Screenshots
26. [x] Lessons learned
27. [x] Architecture decisions

Employer presentation

- [ ] GitHub repository is clean
- [x] No credentials/secrets
- [ ] Meaningful commit history
- [x] Jenkinsfile readable
- [x] Architecture diagram
- [ ] 2–3 minute explanation prepared
- [ ] 10-minute live demonstration prepared

Learning journal history

Every time something goes wrong, record:

Date: 6/9/2026
Technology: Jenkins/chatgpt
Problem: instructions from chatgpt not exactly as in Jenkins
Error: no error message
What I tried: the next best instruction/ closest match
Root cause: possibly chatgpt not informed of version update
Solution: it worked out
What I learned: No technology is perfect.

==========

Date: 8/9/2026
Technology: ssh-myip,
Problem: unable to ssh into aws ec2 from local pc vm, was all ok yesterday
Error:
What I tried: chatgpt went on a diagnosis run while I quietly brainstormed
Root cause: local router switched off overnight, on restart, dhcp assigns new IP address
Solution: visit aws console and change the rules and inbound /32 IP to new IP
What I learned: Chatgpt, though brilliant, lies, implies, hallucinates, can be fooled, is methodical and solves every problem systematically and the long way round to be thorough. I solved this problem in a flash via my own intuition which was switched off. I also learnt that the IP on my Ubuntu on virtualbox is the same as the IP on windows facing the internet. I also had my first practical experience of updating the security group inbound rules live. I was also reminded of what /32 means. My Ubuntu vm has an IP which command # ip a   gives me a local IP, BUT the global IP4 address is the host machine address. From that point on I was switched on while cutting and pasting from chatgpt to my local/aw vm/ubuntu

==========

Date: 3/9/2026
Technology: VIM and terminals, cutting and pasting
Problem: Generally I would cut from terminal to chatgpt input or be copying command from chatgpt to one of two terminals, 1 could be local while the other was AWS. Sometimes I would cut and paste into the wrong terminal or paste output from one terminal incorrectly into chatgpt.
Error:
What I tried: Do less in one sitting to aid concentration
Root cause:
Solution: Focus
What I learned: Supervision of process end to end and an understanding of what of what each command is set to accomplish is a necessary step.

==========

Date: 5/9/2026
Technology: apt update/apt upgrade on ubuntu
Problem: apt update would make suggestions but not update Ubuntu, consequently the s/w installation process was at a standstill
Error:
What I tried: I was at this point clueless. Apt upgrade is a lengthy process so I wanted to avoid going that route.
Root cause:
Solution: Use apt upgrade and bring everything up to speed and inline.
What I learned: apt update will sometimes know what to do but will not do it. The lengthier option apt upgrade is more reliable in this case as a solution

==========

Date: 6/9/2026
Technology: minikube kubectl and sudo install
Problem: minikube, kubectl, docker all installed and working. However they all need sudo to run. When I progressed to automation, it I learnt, required a layered rollback and reinstall Plus some nuanced instruction from chatgpt plus a non root user given root priviliges
Error: as described above
What I tried: bruteforce, and chatgpt, as well as stackoverflow
Root cause: the installation process used root, sudo and was correct but faulty for non sudo non root use
Solution: rollback and reinstall
What I learned: Chatgpt was not informed by me that the Ubuntu machine was using root priviliges. Where I was going was not told to chatgpt. I did not know what I was doing and was just routinely cut and pasting commands. I needed more context input to chatgpt

==========

Date: 5/9/2026
Technology: docker=display
Problem: kubectl, not told what display engine to use
Error:
What I tried:
Root cause:
Solution:
What I learned:

==========

Date: 4/9/2026
Technology: elastic IP on AWS
Problem: Some way into project1, I realised I needed a permanent elastic IP, else projects2 and Project3 would not work.
Error: Over relying on chatgpt for solutions, while not inputting total context
What I tried: Feeding new information about current situation into chatgpt.
Root cause: partly garbage in, garbage out
Solution: change my cost profile, calculate with my head up and not in the sand
What I learned: I still need to apply intelligence and supervise chatgpt, which is brilliant if supervised properly

==========

Date: 29/8/2026
Technology: Memory calculations on local machine WRONG
Problem: My calculations initially were for an i5 with 16Mb of memry. I had forgotten that was the host machine. The vm had been allocated 4 MB plus 100Gb of disk space.
Error: Chatgpt had been informed that the vm was running on 16Gb of ram
What I tried: Go back to chatgpt and update input parameters
Root cause: Limited foresight Plus I started with a small idea that turned into 3 projects. Chatgpt was leading me
Solution: Speak to my mentor and guide chatgpt
What I learned: Even with such brilliant technology at ones fingertips, one can still make glaring and obvious miscalculations. As Confucious said "The brain is a good slave but a bad master." And so is chatgpt

==========

Date: 6/9/2026
Technology: Jenkins update
Problem: I went into Jenkins to add a plugin. Jenkins was intent at that moment to do an update and then a restart
Error:
What I tried: Just add the plugin and get it working. It was an ssh agent that would enable SSH access to AWS EC2
Root cause: timing. Regularity of updates these days. Having not encountered this problem before, I hadn't planned for it
Solution: I accidentally triggered the restart
What I learned: Plan for the unplanned

==========

Date: 12/9/2026
Technology: Ubuntu
Problem: timedatectl producing time which is 16 hours behind windows time.
Error: software updates not working,
What I tried:
Root cause:
Solution: sudo timedatectl set-ntp true; sudo chronyc makestep
What I learned: If vm clock is out of by a margin of 24 hour +, software updates will not take place and apt update will not run.

==========

Date: 6/9/2026
Technology:
Problem:
Error:
What I tried:
Root cause:
Solution:
What I learned:

==========

Date: 6/9/2026
Technology:
Problem:
Error:
What I tried:
Root cause:
Solution:
What I learned:

==========

Date: 6/9/2026
Technology:
Problem:
Error:
What I tried:
Root cause:
Solution:
What I learned:

==========

Date: 6/9/2026
Technology: Jenkins/chatgpt, ssh-myip, aptupdate/apt upgrade
Problem: instructions from chatgpt not exactly as in Jenkins
Error:
What I tried: the next best instruction/ closest match
Root cause: possibly chatgpt not informed of version update
Solution: it worked out
What I learned: No technology is perfect.

Is Project1 reproducable

Only part of project1 is reproducible from this document. The process is lengthy, error prone and time consuming, although since both cost and time are involved, is absolutely necessary. The project was built on two laptops, 1. My home laptop running windows 11 and office and 2. The hme lab, windows with virtualbox sat on top and Ubuntu on top of that. I am still on the lab laptop trying to make mouse bidirectional so I can copy and paste instructions (so as to reduce errors)

Cleanup strategy/ taking project1 down